

PIPELINE → È UNA TECNICA DI IMPLEMENTAZIONE HARDWARE CHE CONSENTE DI SOVRAPPORRE L'ESECUZIONE DI + ISTRUZIONI, RENDENDO IL PROCESSO + VELOCE ED EFFICIENTE → NON È PARALLELISMO!!

IMMAGINIAMO UNA ISTRUZIONE, COME LAVARE I VESTITI:

1) L (LAVARE) 2) A (ASCIUGARE) 3) S (STIRARE) 4) R (RIPORRE I VESTITI)

SENZA PIPELINE → L-A-S-R | L-A-S-R | L-A-S-R

CON PIPELINE → L-A-S-R |
L-A-S-R |
L-A-S-R |
APPENA INIZIA AD ASCIUGARE SI PUÒ LAVARE IL PROSSIMO VESTITO
(VENGONO SOLO SOVRAPPOSTE)

PARALLELISMO → L-A-S-R }
L-A-S-R } TUTTE LE ISTRUZIONI SONO ESEGUITE
L-A-S-R } CONTEMPORANEAMENTE.

LE PIPELINE NON RIDUCONO IL TEMPO PER COMPLETARE UN INCARICO, MA PERMETTONO DI AUMENTARE IL THROUGH PUT, OVVERO LA QUANTITÀ DI LAVORO CHE POSSIAMO COMPLETARE IN UN INTERVALLO DI TEMPO.

OGNI FASE DELLA PIPELINE PUÒ ESSERE VISTA COME UNO STADIO, MENTRE UN'ISTRUZIONE STA PASSANDO ATTRAVERSO UNO STADIO, ALTRE ISTRUZIONI POSSONO "ENTRARE NEGLI STADI PRECEDENTI".

IDEALMENTE OGNI STADIO DOVREBBE RICHIEDERE LO STESSO TEMPO PER COMPLETARE IL PROPRIO COMPITO, MA OVVIAMENTE QUESTO NON È COSÌ E SI CREANO COLLI DI BOTTIGLIA.

$$\text{TEMPO TRA 2 ISTRUZIONI CON PIPELINE} = \frac{\text{TEMPO TRA 2 ISTRUZIONI SENZA PIPELINE}}{\text{NUMERO DI STADI}}$$

TUTTAVIA IL TEMPO DI ESECUZIONE DELLA SINGOLA ISTRUZIONE PUÒ AUMENTARE, A CAUSA DI LATENZE AGGIUNTIVE E PROBLEMI DI SINCRONIZZAZIONE. PUÒ ESSERE QUINDI NECESSARIO UN CICLO DI CLOCK PIÙ LUNGO.

PERCHÉ RISC-V È IDEALE PER LA PIPELINE?

L'ISA DI RISC-V È STATO PROGETTATO PER SEMPLIFICARE L'HARDWARE:

- **LUNGHEZZA FISSA DELLE ISTRUZIONI**: SEMPLIFICA FETCH E DECODE, INVECE IN X86, DOVE LA LUNGHEZZA DELLE ISTR. È **VARIABILE**, LA PIPELINE È **+ COMPLESSA**.
- **UNIFORMITÀ DEI FORMATI**: IL NUMERO DI FORMATI È RIDOTTO E I REGISTRI **SOPR** = **GE** SONO SEMPRE NELLE MEDESIME POSIZIONI DI BIT, VELOCIZZANDO LA LETTURA DURANTE IL DECODE.
- **ARCHITETTURA LOAD/STORE**: SOLO LE ISTRUZIONI **LOAD/STORE** ACCEDONO ALLA MEMORIA, QUESTO PERMETTE DI SEPARARE LO STADIO PER IL CALCOLO DELL'INDIRIZZO DI MEMORIA DALLO STADIO DI ACCESSO ALLA MEMORIA, MANTENENDO LA PIPELINE **+ COMPATTA ED EFFICIENTE**.

ESISTONO SITUAZIONI IN CUI L'ISTRUZIONE SUCCESSIVA NON PUÒ ESSERE ESEGUITA NEL CICLO DI CLOCK IMMEDIATAMENTE SEGUENTE, VENGONO CHIAMATI **HAZARD**, CIÒ COSTRINGE IL PROCESSORE A INSERIRE DEI PAUSEMENTI, OVVERO DEI **CICLI DI ATTESA**, CHIAMATI **STALL** O **BOLLE**.

ESISTONO 3 TIPI DI HAZARD:

- 1) **HAZARD STRUTTURALI**: QUANDO L'HARDWARE NON POSSIENE ABBASTANZA **RISORSE** PER ESEGUIRE SIMULTANEAMENTE ISTRUZIONI CHE SI TROVANO IN **STADI** **DIFFERENTI**.

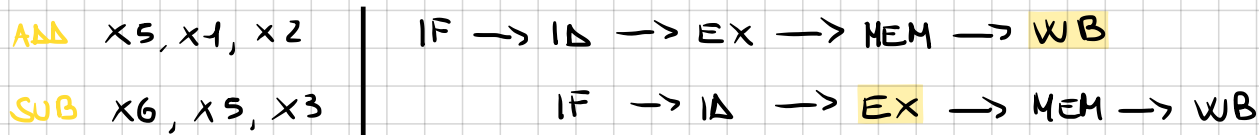
ES: 2 ISTRUZIONI RICHIEDONO L'ACCESSO CONTEMPORANEO ALLA STESSA MEMORIA O ALU NELLO STESSO CICLO

SOLUZIONI IN RISC-V: L'ARCHITETTURA PREVEDE NATIVAMENTE LA SEPARAZIONE TRA MEMORIA DATI E MEMORIA ISTRUZIONI ELIMINANDO L'HAZARD STRUTTURALE DI ACCESSO ALLA MEMORIA NELLO STESSO CICLO

- 2) **HAZARD SUI DATI**: QUANDO 2 ISTRUZIONI CONSECUTIVE DIPENDONO DALLO **STESSO DATO**.

- **RAW**: LA 2° ISTR. LEGGE UN REGISTRO SCRITTO DALLA 1°, MA LA **SCRITTURA** NON È ANCORA AVVENUTA.
- **WAR**: LA 2° ISTR. SCRIVE UN REGISTRO CHE LA PRIMA STA **LEGGENDO**.
- **WAW**: 2 ISTRUZ. SCRIVONO LO STESSO REGISTRO, E L'ORDINE DEVE ESSERE RISPETTATO.

ESEMPIO DI CONDE CRITICO RAW:



QUI SCRIVE IN X5

SOLUZIONI:

MA QUI LO LEGGE GIÀ, LEGGENDO UN VALORE VECCHIO O NON VALIDO

- 1) INTRODUZIONE DI STALLI (BOLLE):** L'HARDWARE RILEVA LA DIPENDENZA E "BLOCCA" TEMPORANEAMENTE L'ISTRUZIONE DIPENDENTE, INSERENDO CICLI DI ATTESA VUOTI, È SEMPLICE DA IMPLEMENTARE MA RIDUCE IL THROUGHPUT.
IN QUESTO MODO SI PERDONO 3 CICLI DI CLOCK, PERCHÉ ID (CHE LEGGE I REGISTRI) E WB (CHE SCRIVE NEI REGISTRI ALLA FINE DELLO STESSO CICLO DI CLOCK)
QUINDI SE IL RISULTATO È SCRITTO DOPO WB, L'ISTRUZIONE SUCCESSIVA DEVO ASPETTARE FINO A QUEL MOMENTO (TRA ID E WB CI SONO 3 STALLI)
COMPRESO
- 2) OTTIMIZZAZIONE VIA COMPILER:** IL COMPILER PUÒ TENTARE DI RIORGANIZZARE LE ISTRUZIONI OPPURE INSERIRE NOOP. TUTTAVIA LE DIPENDENZE SONO TROPPO FREQUENTI E I NOOP CONSUMANO CICLI DI CLOCK.
- 3) PROPAGAZIONE DEI DATI O BYPASS:** LA SOLUZIONE + EFFICIENTE, IL RISULTATO DELL'ISTRUZIONE PRECEDENTE NON VIENE ATTESO FINO AL WB, MA VIENE "INOLTROATO DIRETTAMENTE" DALLO STADIO IN CUI È DISPONIBILE (COME IN EX o MEM) FINO ALL'INGRESSO DELLA ALU PER L'ISTRUZIONE SUCCESSIVA, SENZA STALLI.
IL LIMITE DEL BYPASS: NON RISOLVE TUTTE LE DIPENDENZE DI TIPO RAW, OVERO NON PUÒ RISOLVERE DA SOLO IL CASO IN CUI UNA LOAD SIA SEGUITA DA UN'ISTRUZIONE CHE USI IL REGISTRO APPENA CARICATO. IN QUESTO CASO IL DATO È DISPONIBILE ALLA FINE DI MEM (STADIO 4) MA L'ISTR. SI TROVA ALLA EX (STADIO 3).
USCITA ALU
ENTRATA ALU

PER QUESTO VIENE INSERITA UNA **BOLLA** PER POI FAR INTERVENIRE IL **BYPASS**.

4) **HAZARD SUL CONTROLLO (BRANCH HAZARD)**: SI VERIFICANO CON LE ISTRUZ. DI **SALTO CONDIZIONATO**, PERCHÉ IL PROCESSORE NON SA QUALE ISTRUZIONE **PRELEVA**RE. INFATTI SE LA CONDIZIONE È VERIFICATA ALLO STATO **EX**, IL PROCES. HA GIÀ EFFETTUATO LA **FETCH** DEL **RAMO SBAGLIATO**, COSTRINGENDO A INSERIRE **2 CICLI DI STALLO**.

SOLUZIONE: I MODERNI PROCESSORI USANO SOFISTICATE **TECNICHE HARDWARE DI PREVISIONE DEI SALTI** PER IPOTIZZARE L'ESITO DEL RAMO.

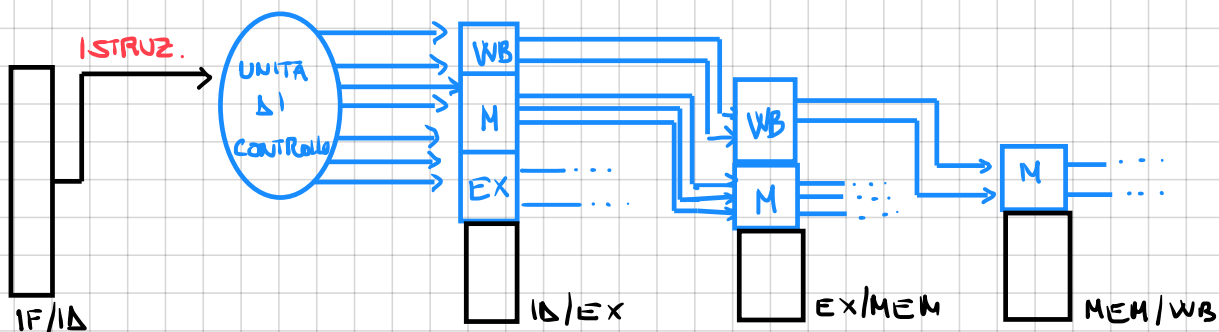
VISTO CHE CI SONO 5 STADI AVREMO BISOGNO DI **4 REGISTRI INTERMEDI**:

- **IF/ID** → IMMAGAZZINA L'ISTRUZIONE APPENA PRELEVATA E CONSERVA IL **PC** (UTILE PER I **SALTI**)
- **ID/EX** → CONSERVA I VALORI LETTI DAI REGISTRI SORGENTE, IL CAMPO IMMENATO ESTESO A 32 BIT E IL **PC**.
- **EX/EM** → CONTIENE IL **RISULTATO DELLA ALU** E IL VALORE DEL SECONDO REGISTRO (PER LA **SCRITTURA STORE**)
- **MEM/WB** → CONSERVA I **DATI LETTI DALLA MEMORIA** O IL **RISULTATO DELLA ALU** PRONTI PER LA **SCRITTURA NEL FILE REGISTER**.

QUESTI REGISTRI SERVONO PER **CONGELARE** LE INFORMAZIONI PRIMA CHE LO STADIO PRECEDENTE LE **SOVRASCRIVA** CON LE INFORMAZIONI DELLO STADIO SUCCESSIVO.

COME È POSSIBILE? BEH I REGISTRI DELLE PIPELINE SONO **EDGE-TRIGGERED**, QUINDI AD ESEMPIO LE PORTE DELLE PIPELINE SI "APRONO" SOLO AL CAMBIAMENTO DI STATO DEL CLOCK, QUESTO PERMETTE DI FAR AVANZARE IL DATO TRA UNO STADIO E L'ALTRO SOLO AD ESEMPIO AL **FRONTE DI SALTA** (0 → 1). (PERMETTENDO L'AVANZAMENTO TRA I DIVERSI STADI DEL **REGISTRO DI DESTINAZIONE**).

COME INFLUISCE TUTTO CIÒ NELLA GENERAZIONE DEI SEGNALE DI CONTROLLO?



OVVIAMENTE ANCHE I SEGNALI DI CONTROLLO DEVONO ESSERE PRESERVATI TRA GLI STADI, CHE VENGONO RAGGRUPPATI IN:

- EX → ALUOP, ALUSAC → VENGONO CONSUMATI NELLO STADIO EX E NON VENGONO PROPAGATI OLTRE.
- M → BRANCH, MEMREAD, MEMWRITE → VENGONO MANTENUTI NEL REGISTRO ID/EX, VIAGGIANO INTATTI FINO ALLO STADIO MEM.
- WB → MEMTOREG, REGWRITE → VENGONO PROPAGATI DAI REGISTRI ID/EX E EX/MEM PER ESSERE USATI NELLO STADIO WB.